南方科技大学

LAB2: PyTorch

COE201, Spring 2026

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

Vectorization and Tensors 1

Why Vectorization? 2

What is PyTorch? 6

PyTorch Overview 7

Computational Graph and Autograd . . 9

Understanding Autograd 10

Build a Neural Network with PyTorch 15

Neural Network Basics 16

Lab Tasks 21

MNIST Dataset & MLP Architecture 22

Lab 2 Exercises 24

Optional Exercises 25

Glossary 27

Other References 28

Vectorization and Tensors

Why Vectorization?

The Problem with Python Loops

```python
# Slow: Python loop with type checking at each step
result = []
for i in range(1000000):
    result.append(a[i] + b[i])

# Fast: Vectorized operations (C-level, contiguous memory)
result = a + b  # 100x+ faster!
```

Why the difference?

- Python loops: CPU synchronous execution, processing elements one by one.
- Vectorized operations: Hardware-level parallelism for simultaneous execution.

Why Vectorization?

Even Better on GPU

GPUs have **thousands of cores** designed for parallel computation:

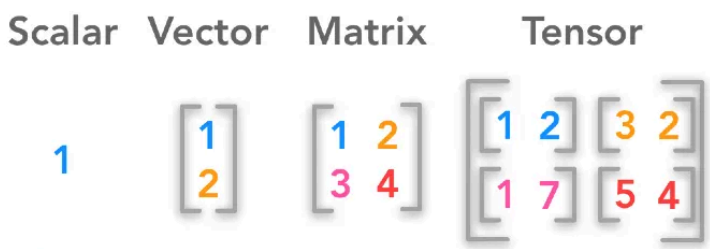
	CPU	GPU
Cores	8-16	**thousands**
Best for	Sequential tasks	Parallel tasks
Example	Web browsing, Office apps	Matrix multiplication

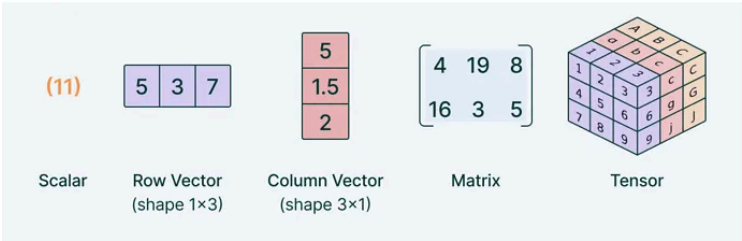
Vectorized code $\longrightarrow$ GPU can process thousands of elements **simultaneously**

Why Vectorization?

What is a Tensor?

To leverage vectorization, we need a **data structure** that stores data in contiguous blocks of memory.





Tensor dimensions:

- **Scalar** (0-D): single value
- **Vector** (1-D): list of values
- **Matrix** (2-D): table of values
- **Tensor** (N-D): N-dimensional array

Tensor = multi/any-dimensional array designed for vectorized operations.

Why Vectorization?

Tensor Operations

```python
# Creating tensors
x = torch.zeros(3, 3)        # All zeros
x = torch.ones(3, 3)         # All ones
x = torch.randn(3, 3)        # Random normal distribution
x = torch.arange(0, 10, 2)   # [0, 2, 4, 6, 8]

# Basic operations (element-wise)
a = torch.tensor([1., 2., 3.])
b = torch.tensor([4., 5., 6.])
print(a + b)      # tensor([5., 7., 9.])
print(a * b)      # tensor([4., 10., 18.])
print(a @ b)      # tensor(32.) - dot product

# Reshaping
x = torch.arange(12)
x = x.reshape(3, 4)   # 3 rows, 4 columns
x = x.view(2, 6)      # Different shape, same data
```

What is PyTorch?

PyTorch Overview

PyTorch: A Deep Learning Framework

PyTorch is a tensor library with two key features:

1. **GPU Acceleration** - Run computations on NVIDIA (and others) GPUs
2. **Automatic Differentiation** - Compute gradients automatically

```python
import torch

# Check if GPU is available
print(torch.cuda.is_available())  # True if GPU available

# Move tensor to GPU
x = torch.randn(1000, 1000)
x_gpu = x.to('cuda')  # or x.cuda()

# Computation happens on GPU
y_gpu = x_gpu @ x_gpu.T
```

PyTorch Overview

NumPy vs PyTorch

Both are **vectorization libraries** for fast numerical computation. PyTorch adds features for deep learning:

Feature	NumPy	PyTorch
Vectorization	✅	✅
GPU Support	❌	✅
Auto Gradient	❌	✅
Neural Network Modules	❌	✅ (`nn.Module`)
Optimizers	❌	✅ (`torch.optim`)

Computational Graph and Autograd

Understanding Autograd

南方科技大学

Backpropagation: The Core of Deep Learning

How do neural networks learn? In one iteration:

1. **Forward pass**: Input $\longrightarrow$ Network $\longrightarrow$ Prediction
2. **Compute loss**: Measure how far off the prediction is from the truth.
3. **Backward pass**: Compute the gradients of the loss with respect to all weights.
4. **Update weights (Optimization)**: Adjust weights in the opposite direction of the gradient to minimize the loss.

Backpropagation (step 3) is the algorithm that makes deep learning possible.

- It efficiently computes gradients for **millions** of parameters.
- Without it, training deep neural networks would be computationally impossible.

Understanding Autograd

Chain Rule: The Math Behind Backprop

To compute gradients through a chain of operations, we use the **chain rule**:

$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

Example: Given $y = z^2$, $z = x + 2$. If $x = 3$, calculate $\frac{dy}{dx}$.

Forward pass:

- Given $x = 3$
- Then $z = x + 2 = 5$
- Finally $y = z^2 = 25$

Backward pass:

- $\frac{dy}{dz} = 2z = 10$
- $\frac{dz}{dx} = 1$
- $\frac{dy}{dx} = 10 \times 1 = 10$

How can we track these operations and compute gradients automatically?

Understanding Autograd

Computational Graph

A **computational graph** is a Directed Acyclic Graph (**DAG**) that tracks a sequence of operations.

Same example: input x, then $z = x + 2$, and output $y = z^2$.

```
x ─┐
   ├─► (+) ─► z ─► (^2) ─► y
2 ─┘
```

So now we know how the data flows:

- **Forward pass**: Build the graph, input $x = 3$, and compute $z = 5$, $y = 25$.
- **Backward pass**: Traverse the graph in reverse, applying the chain rule at each node.

This same principle scales to neural networks with **millions** of operations!

Autograd in Action

Same example in PyTorch: $y = z^2$, $z = x + 2$, $x = 3$

```python
import torch

# Create tensor with gradient tracking
x = torch.tensor(3.0, requires_grad=True)

# Forward pass: PyTorch builds the computational graph automatically
z = x + 2       # z = 5
y = z ** 2      # y = 25

# Backward pass: Compute gradients
y.backward()

# Gradient is stored in .grad attribute
print(x.grad)  # dy/dx = 10.0 ✓ (same as our manual calculation!)
```

Detach and No Grad

```python
# .detach() - Remove tensor from computation graph
y = w * x
y_detached = y.detach()  # No longer tracks gradients

# torch.no_grad() - Disable gradient computation (inference)
with torch.no_grad():
    y = model(x)  # Faster, less memory

# Common use: updating weights manually
w.data -= learning_rate * w.grad  # Don't track this update!
w.grad.zero_()  # Reset gradients for next iteration
```

Build a Neural Network with PyTorch

Using nn.Module

```python
import torch
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
# Create model
model = SimpleNet(784, 128, 10)
```

Using nn.Sequential (Simpler)

```python
# For simple feed-forward networks (e.g., MNIST digit classification)
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
)

# Forward pass
import torch
x = torch.randn(32, 1, 28, 28)  # Batch of 32 images, 1 channel, 28x28
output = model(x)
print(output.shape)             # torch.Size([32, 10])
```

Optimizers: Gradient Descent (GD) vs SGD

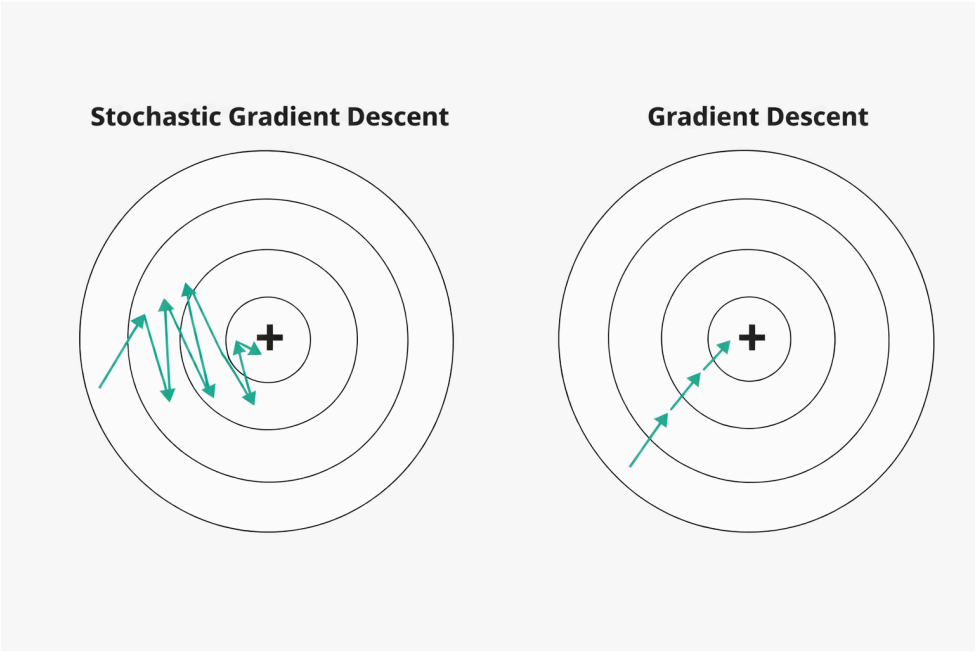
Optimization algorithms determine **how** to update the model weights using their computed gradients to minimize the loss.

- **Gradient Descent (GD)**: Calculates the gradient using the **entire training dataset**.
  - **Pros**: Provides a stable and accurate gradient direction.
  - **Cons**: Extremely slow; computes over the full dataset just to make a single update.
- **Stochastic Gradient Descent (SGD)**:
  - Originally refers to updating weights using just **a single sample** at a time ($\eta \nabla L(x_i, y_i)$).
  - **Pros**: Enables very fast updates.
  - **Cons**: High variance; the gradient direction is noisy and fluctuates significantly.

Neural Network Basics

Tradeoff of optimizer in a very simple example:

- SGD: Fast but not so accurate on each step
- GD: Slow but accurate on each step



Mini-Batch SGD & PyTorch Optimizer

In modern Deep Learning, "SGD" almost always refers to **Mini-Batch SGD**.

- Divides the dataset into small batches (e.g., 32, 64, or 128 samples).
- Combines the **speed/memory efficiency** of SGD with the **stability** of full GD.
- In PyTorch, a `DataLoader` yields these mini-batches automatically!

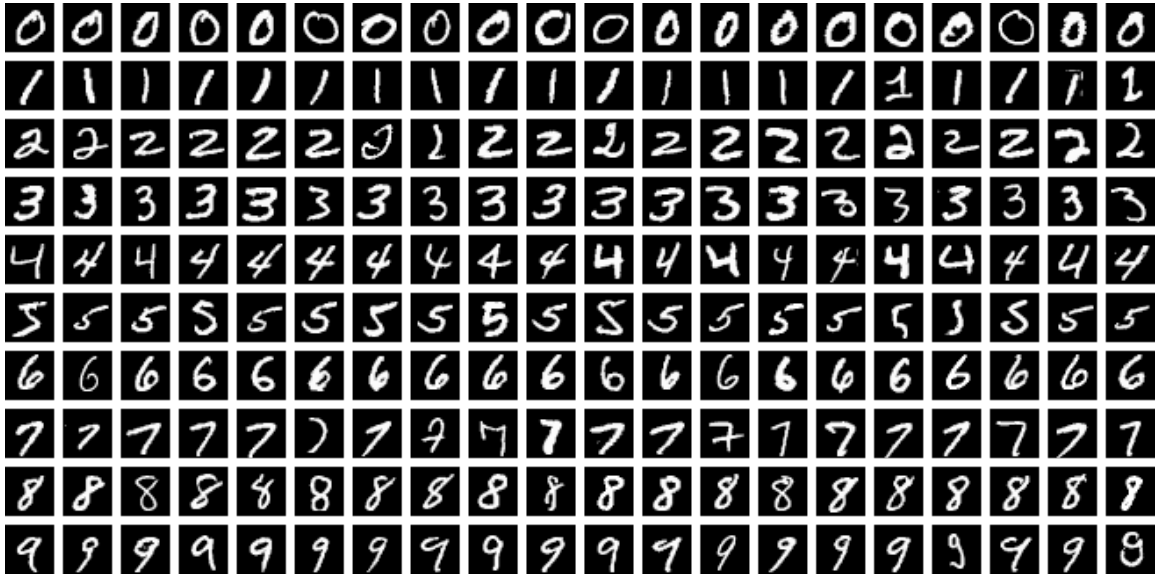
A simplest train loop with PyTorch Python

```python
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
# Training loop with Mini-Batch SGD
for images, labels in train_loader: # train_loader yields batches of 32
    outputs = model(images)
    loss = criterion(outputs, labels) # Loss over the mini-batch
    optimizer.zero_grad()  # 1. Clear previous gradients
    loss.backward()        # 2. Compute gradients for this batch
    optimizer.step()       # 3. Update weights: w = w - lr * grad
```

Lab Tasks

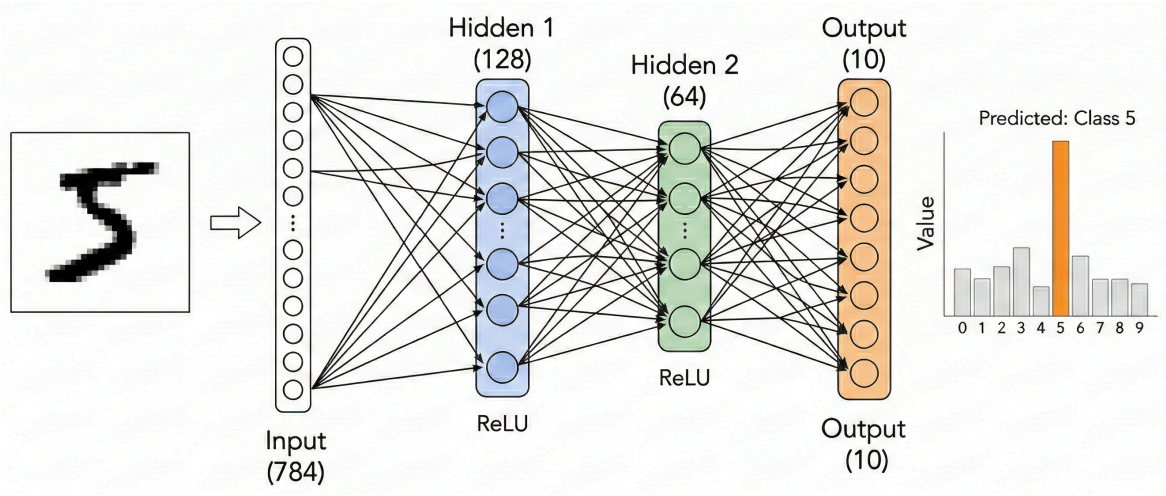
MNIST Dataset & MLP Architecture

- **MNIST Dataset**: A classic dataset of handwritten digits (0-9).
  - 60,000 training images, 10,000 testing images.
  - Each image is a 28×28 grayscale pixel matrix.
  - Task: Train a Neural Network to classify the image into one of the 10 digits (0-9).



MNIST Dataset & MLP Architecture

- **Multi-Layer Perceptron (MLP)**:
  - **Input Layer**: Flattens the 28×28 image into a 1D tensor of length 784.
  - **Hidden Layers**: Fully connected linear layers followed by activation functions.
  - **Output Layer**: Final linear layer outputting 10 values (logits), representing the scores for each digit class.



Lab 2 Exercises

The sections you need to implement are marked with ### TODO in the code.

Task 1: Manual Gradient Descent (`lab2_manual_gd.py`)

- Target: $y_p = w * x + b$; Loss: MSE $= \frac{1}{N}\sum\left(y_p - y_t\right)^2$
- Parameters Update: $w = w - \eta \cdot \nabla w$, $b = b - \eta \cdot \nabla b$
- Tips:
  1. Perform `.backward()` and update weights inside a `torch.no_grad()` context.
  2. Use `.grad` to retrieve the gradients after calling `.backward()`.
  3. Use `.zero_()` to clear the gradients before the next iteration.

Task 2: Build a 3-Layer MLP for MNIST (`lab2_mnist_mlp.py`)

- Build a feed-forward model using `nn.Sequential`.
- Implement a complete training loop with an optimizer.

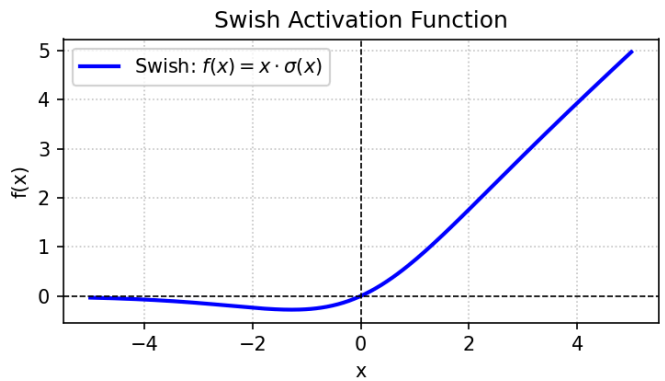
Optional Exercises

Task 3 (Optional): Custom Autograd Function (`lab2_opt1_custom_autograd.py`)

- Manually implement the Swish Activation function and its derivative.

Swish: $f(x) = x \cdot \text{sigmoid}(x) = \frac{x}{1+e^{-x}}$

Hints: Derivative of sigmoid:
$\text{sigmoid}'(x) = \text{sigmoid}(x)(1 - \text{sigmoid}(x))$



Thinking: Why might we need to implement the backward pass manually if PyTorch can do it automatically with autograd?

Optional Exercises

Task 4 (Optional): Polynomial Regression & Optimizers (`lab2_opt2_poly_regression.py`)

- Construct a non-linear prediction function (e.g., $y = a + bx + cx^2 + dx^3$) using tensors with `requires_grad=True`.
- Implement the training loops for different optimization strategies!
  - Fill in the TODOs for full GD, Mini-Batch SGD, and Stochastic GD.
  - **Hint**: You may need to adjust the learning rate for each optimizer to achieve stable training.

Glossary

Quick reference for common Deep Learning terminology:

- **Batch Size**: The number of training samples processed in one forward/backward pass (refer to the Mini-Batch SGD & PyTorch Optimizer slide).
- **Iteration (Step)**: A single update of the model's parameters (1 iteration = processing 1 batch).
- **Epoch**: One complete pass through the **entire** training dataset.
  - If you have 60,000 images and a batch size of 60, it takes 1,000 **Iterations** to complete 1 **Epoch**.
- **Learning Rate (η, LR)**: A tiny scalar (e.g., 0.01) that determines how large the steps are when updating the weights.

Other References

Some useful resources:

- Calculus Basics (Gradient, Derivative, Chain Rule): link
- Backpropagation and Autograd: link
- Gradient Descent: link

Thanks for Listening!